

Research Plan for a Professional Web-Based Text Editing Application

Executive summary

The most important architectural decision is to **avoid building directly on raw** `contenteditable` **plus** `execCommand()` **as the product's primary editing engine**. The web platform still exposes `contenteditable`, `beforeinput`, Selection, Clipboard, and newer proposals like EditContext, but browser behavior for editing remains inconsistent, and `execCommand()` is deprecated. In practice, production editors that need reliable undo/redo, schema validation, collaboration, paste normalization, and rich formatting use a structured editor runtime above the DOM rather than trusting browser editing behavior alone. ¹

For a **net-new, professional, Word-like web editor** where the editor itself is part of the product advantage, the strongest strategic choice is **Tiptap on top of ProseMirror**, with **Yjs for collaboration**, **React for the application shell**, **IndexedDB plus a service worker for offline resilience**, and a **server-side conversion layer** for DOCX/PDF import and export. Tiptap gives the UI freedom of a headless editor; while ProseMirror gives the hard parts: schemas, transactions, plugins, and collaboration primitives. Tiptap's current product surface also explicitly includes collaboration, comments, tracked changes, versioning, file conversion, pages, invisible characters, line-height, font family, font size, table of contents, and AI tooling. ²

There is one meaningful caveat: if the goal is **shortest path to enterprise-grade Word features right now**, especially **track changes, revision history, Word import/export, and pagination**, **CKEditor 5** is the more mature out-of-the-box option. Its architecture is explicitly model/controller/view, it ships a schema system, standalone or collaborative track changes, revision history, pagination, Word import, and Word export, and it already documents the integration points among those features. Put simply: **Tiptap/ProseMirror is the best platform to build a differentiated editor product; CKEditor 5 is the best off-the-shelf shortcut when feature completeness matters more than UI freedom.** ³

The rest of this report assumes the primary recommendation is **Tiptap + ProseMirror + Yjs**, with **CKEditor 5** kept as the principal alternative if the business chooses to optimize for speed-to-market over architectural control. ⁴

Product and interaction model

A professional editor should use a **hybrid interface**, not a single interaction pattern. The best desktop layout is a **top ribbon** for high-frequency formatting and layout operations, a **document canvas** in the center, a **left panel** for thumbnails or outline/navigation, a **right inspector** for contextual properties, and a **status bar** for zoom, page stats, language, review mode, and collaboration state. Floating toolbars and slash commands are useful, but they should complement the ribbon rather than replace it: floating controls are ideal for inline text actions, while slash commands are ideal for insertions such as tables, images,

headings, templates, footnotes, or blocks. Tiptap's extension catalog itself reflects this split, offering bubble menus, floating menus, slash command patterns, table-of-contents support, and page-based editing features rather than assuming one universal UI. ⁵

For **desktop**, keep the ribbon visible and allow tabbed groups such as Home, Insert, Layout, Review, and View. For **tablet**, collapse the ribbon into horizontally scrollable groups and move the inspector into a slide-over panel. For **mobile**, prioritize a document-first view with a compact top bar, a bottom formatting tray, and contextual tool sheets; a full Word-style ribbon does not scale well to phone widths. The system should also auto-switch comment balloons and suggestion controls between sidebar and inline modes depending on available width, which is the same responsive pattern CKEditor documents for track changes and comments. ⁶

Accessibility cannot be bolted on later. The ribbon should use the ARIA `toolbar` pattern so screen-reader users can understand grouped controls and keyboard users do not get trapped in hundreds of tab stops. Focus appearance, target sizing, and drag alternatives matter for this type of dense UI, especially because an editor mixes mouse-heavy and keyboard-heavy interaction. WCAG 2.2 and the WAI authoring guidance make those requirements explicit. Native spellcheck can be enabled where appropriate, and voice input can be layered on with the Web Speech API as an optional feature rather than as the only path to dictation. ⁷

The best UX improvements beyond a basic ribbon are practical rather than flashy: a real ruler, visible page boundaries when pagination is on, invisible-characters toggle, style previews in the style gallery, keyboard-first command search, intelligent paste options, predictable selection preservation, and clear review states such as Editing, Suggesting, and Viewing. Tiptap exposes invisible characters, selection preservation, pages, and style-oriented text extensions directly in its ecosystem, which is a good sign for this model. ⁸

Document model and editing systems

The editor should treat **document data as structured JSON, not HTML**, and then derive HTML, DOM, DOCX, or PDF from that canonical model. ProseMirror's schema model and CKEditor 5's model/schema architecture both point in this direction: nodes define allowed structure, marks define inline formatting, and converters translate between the internal model and external data or rendered views. This is the right foundation for preserving formatting reliably, validating pasted content, supporting document-wide styles, and round-tripping collaborative edits. ⁹

A strong **data model recommendation** is to separate the document into four layers. First, a **content tree**: paragraphs, headings, tables, lists, images, sections, page breaks, bookmarks, comments anchors, and other nodes. Second, **inline marks**: bold, italic, underline, strike, subscript, superscript, font family, font size, color, highlight, and typographic effects. Third, **paragraph and section attributes**: alignment, indentation, line-height, spacing before/after, borders, shading, columns, margins, orientation, page size, line numbering, and hyphenation settings. Fourth, a **style registry**: named styles such as Normal, No Spacing, Heading 1, Heading 2, Quote, Title, Subtitle, Code Block, and custom styles. This separation maps naturally onto schema-based runtimes such as ProseMirror, CKEditor 5, Lexical node trees, and Quill's distinction between content and formats. ¹⁰

For **clipboard and basic editing**, use the browser Clipboard API when possible, but always keep event-based fallbacks for `copy`, `cut`, `paste`, and drag-and-drop operations. The platform only allows

clipboard access in secure contexts; read access often requires recent user activation and may trigger permissions or prompts depending on the browser. The API is also not available in Web Workers, so paste normalization and conversion must happen on the main thread or via message-passing after the initial event. Rich-text editors that do this well always define MIME-type priority and normalization rules. Lexical, for example, serializes selections to `text/plain`, `text/html`, and its own JSON MIME type, and its rich-text import pipeline prioritizes structured editor data, then HTML, then plain text. Quill's clipboard module parses pasted HTML into its Delta model using matchers, and TinyMCE's PowerPaste cleans Word and Excel HTML, optionally handling pasted images. ¹¹

Format Painter should copy **semantic formatting tokens**, not computed CSS text. A source selection should be read into a “format bundle” that includes inline marks plus paragraph properties, with a mode switch for “character only” versus “character + paragraph.” Applying that bundle should be a single editor transaction so the action is undoable as one step. TinyMCE's premium Format Painter is a useful benchmark here because it explicitly copies formatting and supports repeated application, which matches user expectations from Word. ¹²

Font and typography tools belong in the mark layer. Bold, italic, underline, strike, superscript, subscript, font family, size, color, and highlight should be direct inline marks or text attributes. Decorative effects such as shadow, outline, glow, small caps, and letter spacing should be implemented either as a constrained “text effects” mark or as a group of safely exportable attributes, because many downstream formats will not preserve arbitrary CSS. The safest rule is: if a visual effect cannot round-trip to HTML, DOCX, and PDF with predictable results, expose it only behind a compatibility flag. Tiptap's extension surface already separates font family, font size, text color, background color, and typography-related behavior into explicit extensions, which aligns with this approach. ¹³

Change case looks simple, but it is usually implemented poorly in multilingual editors. Use locale-aware segmentation and boundaries rather than ASCII heuristics. `Intl.Segmenter` exists precisely for locale-sensitive grapheme, word, and sentence segmentation; it is a better base for title case and sentence case than naïve regex splitting. ¹⁴

Paragraph tools should be paragraph- or block-level attributes, not inline formatting. Bullets, numbering, multilevel lists, indent/outdent, alignment, spacing, shading, borders, and columns all belong to block or section nodes. To prevent broken list nesting, define one canonical list structure in the schema and normalize all pasted or user-produced variants into it. This is where ProseMirror schemas, Slate normalization, and CKEditor post-fixers all point to the same lesson: the editor must continuously repair invalid structures rather than hoping the DOM stays valid. ¹⁵

For **styles**, do not store “Heading 1” as a blob of repeated CSS on every node. Store a style reference such as `paragraph.styleId = "heading1"` plus any direct overrides. Then resolve the final formatting as `theme defaults → style properties → local overrides`. That makes document-wide updates fast and predictable: changing the definition of Heading 1 updates every node that references it. It also keeps import/export cleaner, because style mapping to DOCX or HTML classes becomes explicit rather than inferred. Mammoth's docx-to-HTML conversion is a strong reminder that semantic styles are much easier to preserve than exact visual formatting; it intentionally maps styles like Heading 1 semantically and warns that HTML and DOCX do not align perfectly for complicated styling. ¹⁶

Find and replace for large documents should use a **plain-text projection plus offset map** instead of searching the rendered DOM. Build and maintain an incremental text index per block, section, comment body, and table cell. Run complex or regex searches in a Web Worker so the UI stays responsive, then map matches back to document positions and display them with decorations or the Custom Highlight API. Selection APIs and document `selectionchange` events are still useful for the visible UX layer, but they should not be the source of truth for search offsets. ¹⁷

Page setup and page layout should be stored in section nodes or section attributes: margins, page size, orientation, columns, line numbering, hyphenation, and explicit breaks. Pages and page previews are then a **layout view derived from the content model**, not the primary data model. Tiptap Pages explicitly models page boundaries, formats, margins, headers, footers, page breaks, zoom, footnotes, and endnotes. CKEditor's pagination feature makes the same distinction by showing where exported page breaks would land rather than pretending the browser DOM is itself a paged document engine. Both products also openly document pagination caveats, especially around browser differences and export accuracy. ¹⁸

For **hyphenation**, use native CSS `hyphens` with correct language tagging first, and only add a polyfill such as Hyphenopoly when target browsers or languages need it. For **line numbers**, treat them as a layout feature, not stored text. CSS counters are useful for numbering structural elements, but true Word-style line numbers depend on actual line layout and pagination, so they belong in the page layout engine. ¹⁹

Object arrangement should live in a parallel object layer. Images, charts, text boxes, shapes, and embedded media should be represented as atomic nodes in the document model, but rendered through an overlay system that handles drag, resize, rotate, z-order, and text wrap. Pointer Events give a unified mouse/touch/pen model, CSS transforms handle rotate/scale/translate, and ResizeObserver is useful when object dimensions change and layout needs recomputation. This is also why floating objects should store anchor information, wrap mode, and z-index as object attributes rather than as ad hoc CSS fragments. ²⁰

Framework and application architecture

The editor framework comparison is clearer when aligned to the actual product goal. **ProseMirror** is the most important low-level runtime in this space because it gives explicit schemas, transactions, plugins, and a collaboration module that works with a central authority or can be paired with Yjs. It is powerful but intentionally low-level. **Tiptap** is the best product-oriented layer above it because it keeps the ProseMirror power while adding a headless, extension-first API and a growing set of production features around collaboration, comments, file conversion, and pages. ²¹

CKEditor 5 is the strongest alternative when you want an editor platform that already thinks like a document processor. Its engine is explicitly model/controller/view; it maintains separate editing and data views; it has a schema system and conversion pipeline; and its product surface includes track changes, revision history, pagination, Word import/export, and comments. For many teams, that makes CKEditor the "enterprise shortcut." The tradeoff is architectural openness and UI freedom: you can still customize it heavily, but it is less naturally headless than Tiptap's design philosophy. ³

Lexical is impressive for immutable editor state, serializable node trees, performant updates, HTML conversion utilities, and Yjs collaboration guidance. It is an excellent choice for CMS editors, publishing tools, comments, and social or app-embedded rich text. But for a Word-class product, Lexical gives you

more of the primitives than the word-processing subsystems; pagination, style galleries, print layout, headers/footers, and review workflows would be more of your job. ²²

Slate remains highly flexible, and its normalization model is conceptually attractive, but it is still more “editor toolkit” than “document processor,” which means more assembly and more ongoing ownership for office-like behavior. **Quill** is coherent and pleasant for medium-complexity rich text, with a clean clipboard module, toolbar module, and Delta model, but it is less suited to deeply structured paginated documents with review workflows. **TinyMCE** is mature, broad, and has strong premium features such as PowerPaste, Format Painter, comments, and revision history, but its real-time collaboration offering is no longer available for new customers. **Editor.js** is appealing for block content, but even advocates acknowledge its weak real-time collaboration story, and its block-first mental model is a poor fit for Word-style free-flowing paginated authoring. ²³

The recommended **frontend stack** is **React + Next.js** for the application shell, because React’s ecosystem fits Tiptap/Lexical/Slate well and `useSyncExternalStore` is the right primitive when the editor itself is an external mutable store that must stay consistent under concurrent rendering. Use **Web Workers** for expensive searches, imports, diffing, and pagination assistance; **IndexedDB** for local document snapshots, asset blobs, and offline queues; and a **service worker** for offline boot and cache control. Next.js now documents PWA support directly, which lowers the cost of building a document editor that remains usable without the network. ²⁴

A practical **technical architecture diagram** looks like this:

```
Browser Client
  React / Next.js shell
    Ribbon
    Inspector panels
    Comments / history / thumbnails / status bar
  Editor runtime
    Tiptap extensions
    ProseMirror state, schema, commands, transactions
  Collaboration adapter
    Yjs document
    Presence / awareness
    WebSocket or managed provider
  Layout services
    Pagination
    Ruler / columns / line numbering
    Object wrapping / z-order
  Local resilience
    IndexedDB document cache
    Service worker asset + shell cache
  Background processing
    Search worker
    Import/export worker helpers
```

Backend

- Auth / ACL / sharing
- Document API
- Versions / comments / suggestions API
- Asset storage
- Conversion service
 - DOCX import
 - DOCX export
 - PDF export
- Search / analytics / audit logs

That split follows the same architectural themes visible in ProseMirror's plugin model, CKEditor's separated model and views, Yjs' provider architecture, Lexical's "external store" style editor state, and the browser's own offline APIs. ²⁵

Collaboration, conversion, and security

For **real-time collaboration**, use **Yjs** unless there is a strong reason not to. Yjs is a CRDT-based system with provider-based networking, awareness/presence support, and documented offline options. Tiptap's collaboration features are already built around this model, and Lexical's own collaboration FAQ recommends treating the Yjs model as the source of truth when offline recovery and sync matter. ProseMirror's native collab plugin is still useful, but its official model assumes a central authority determining step order; that is workable, but Yjs gives a more natural offline-first path. ²⁶

For **comments, track changes, and version history**, keep three separate concepts. **Comments** are conversational annotations attached to ranges or objects. **Tracked changes** are suggested edits that can be accepted or rejected. **Version history** is a sequence of snapshots or named revisions of the whole document. CKEditor's docs make this distinction very clearly: track changes can operate standalone or with real-time collaboration, and revision history is a different capability focused on document versioning over time. Tiptap's current surface also reflects the same separation through Comments, Snapshot/versioning, compare snapshots, and tracked changes. ²⁷

The **review model recommendation** is to store comments and suggestions as **annotation data**, not as baked HTML spans whenever possible. CKEditor's markup example is useful as a transport representation, but for your own source of truth, range-anchored annotations are easier to query, permission, diff, and export. Suggestion mode should behave like Word: inserted text and deletions are represented as suggestions, the sidebar is authoritative, and accept/reject acts transactionally. ²⁸

For **DOCX import/export**, the key decision is whether you want **high fidelity** or **semantic interoperability**. For semantic import, Mammoth is useful because it converts DOCX to clean HTML based on styles, but it explicitly warns that DOCX and HTML have a large structural mismatch and that complicated documents will not convert perfectly. For high-fidelity office workflows, commercial converters are usually better. Both CKEditor and Tiptap now position DOCX conversion as a product feature, and CKEditor explicitly supports importing comments and tracked changes from Word documents when the related editor features are present. ²⁹

For **PDF export**, there are three viable tiers. The most basic is **browser print styles** with `break-before` / `break-after` and paged CSS. The flexible open-source option is **Paged.js**, which exists specifically to paginate HTML in the browser and generate PDF-oriented output. The highest-fidelity option is a **server-side conversion pipeline** that takes your canonical document plus styles and renders PDF in a controlled environment. If you need Word-like pagination, headers/footers, and consistent export across browsers, the server-side path is usually the right long-term choice. CKEditor's own pagination/export documentation reflects exactly how fragile browser-based pagination can be, especially across Firefox and Safari, and recommends careful handling of page breaks when exporting to Word. ³⁰

Security should assume that **paste, imports, and uploads are hostile by default**. OWASP recommends HTML sanitization for WYSIWYG content and specifically points to DOMPurify for that purpose. DOMPurify is a mature sanitizer for HTML, SVG, and MathML. On top of sanitization, modern browsers now support **Trusted Types**, and MDN documents `require-trusted-types-for` as a CSP directive that restricts dangerous DOM XSS sinks such as `innerHTML`. This combination—sanitize, produce trusted HTML, and lock down sinks—is the right browser-side defense for rich-text editors. ³¹

File upload security needs its own controls. OWASP's file upload guidance is explicit: allow only safe extensions, validate actual file types instead of trusting `Content-Type`, rename files, limit size, require authorization, store outside the webroot when possible, and scan or reconstruct risky document formats such as DOCX or PDF. Also remember that the HTML `accept` attribute is only a hint to the browser, not real validation. ³²

For **encryption and transport**, the browser-facing application should run only in secure contexts so that powerful APIs like Clipboard and service workers remain available and protected from man-in-the-middle exposure. At-rest protection should follow standard cryptographic storage controls on the server side, especially for document bodies, revisions, comments, and upload metadata. ³³

Roadmap, testing, and final recommendation

A realistic **implementation roadmap** has four stages. **Phase one** is a high-quality single-user editor: schema, transactions, toolbar, style gallery, safe paste, lists, tables, images, find/replace, native spellcheck, autosave, local persistence, and HTML export. **Phase two** adds word-processor layout features: sections, page setup, pagination preview, headers/footers, footnotes/endnotes, table of contents, bookmarks, and PDF/DOCX export. **Phase three** adds collaboration and review: Yjs sync, presence, comments, suggestion mode, revision history, permissions, and audit logs. **Phase four** adds platform leverage: templates, AI assist, voice typing, grammar services, advanced object layout, and enterprise admin/reporting. That sequencing matches the maturity curve visible in Tiptap Pages/collaboration and CKEditor's review and conversion features: layout and review are hard enough that they deserve separate investment phases. ³⁴

The **MVP feature list** should be narrower than many teams think: rich text marks; paragraph formatting; list handling with normalization; image insertion; table editing; intelligent paste with plain-text option; find/replace; named styles; comments; autosave; offline cache; and export to HTML/PDF. That is enough to support most serious drafting workflows without prematurely locking the team into page-faithful DOCX round-tripping or full Word review parity. The **advanced feature list** should then include pagination, section layout, headers/footers, footnotes/endnotes, track changes, version history, tables of contents,

import from DOCX, export to DOCX, floating objects with wrap modes, AI assistance, and enterprise permission models. ³⁵

The best **libraries and APIs** for this stack are these: **Tiptap + ProseMirror** for the editor runtime; **Yjs** for collaboration and offline-friendly sync; **IndexedDB** and **service workers** for resilience; **DOMPurify** and **Trusted Types** for paste and render safety; **Paged.js** or a server exporter for print-oriented PDF; **Mammoth** only for lower-fidelity semantic DOCX import; and **Playwright**, **Vitest**, and **axe-core** for testing. Each of those tools is strong in its lane, but they should not all sit on the critical path from day one. ³⁶

The **performance strategy** should focus on batching and isolation. Keep typing on the main thread fast by using the editor runtime's own transaction batching, projecting search and diff work to Web Workers, and persisting large local data in IndexedDB rather than `localStorage`. Keep the React shell subscribed to editor state through `useSyncExternalStore` or an equivalent pattern so concurrent rendering does not tear the UI. Recompute pagination incrementally around changed regions, not the full document, and avoid DOM virtualization that breaks selection unless you have a very mature mapping layer. ³⁷

The **testing strategy** should mix deterministic unit tests with cross-browser end-to-end tests. Unit-test every command, transform, normalization rule, and import/export mapper. End-to-end test typing, selections, paste, drag/drop, comments, suggestion mode, and pagination navigation in Chromium, WebKit, and Firefox, because the browser differences are real and documented. Add accessibility automation with axe-core, and maintain regression suites of “nasty paste” samples from Word, Google Docs, HTML email, spreadsheets, and hostile HTML. ³⁸

The biggest **risks and limitations** are not the obvious ones. The hardest problems are **round-tripping fidelity across JSON/HTML/DOCX/PDF**, **predictable pagination in all browsers**, **keeping list and style normalization invisible to users**, and **avoiding XSS and broken markup from paste/import pipelines**. This is exactly why the recommendation is not “just use HTML”: production document editors are data engines, conversion engines, layout engines, and UI systems at once. ³⁹

The **final recommendation** is: build the product on **Tiptap + ProseMirror + Yjs**, store a **schema-validated JSON document model** with a **separate style registry and annotation layers**, use **Pages** or an **equivalent page-layout subsystem** for Word-like layout, and treat **DOCX/PDF as external exchange formats handled by a dedicated conversion pipeline**, not as your canonical model. If the business needs the fastest credible route to enterprise review and Word interoperability, license **CKEditor 5** instead. But if the product's editor is meant to be differentiated, extensible, and deeply integrated into your own workflow, the Tiptap/ProseMirror architecture is the better long-term foundation. ⁴⁰

¹ ContentEditable

https://www.w3.org/TR/content-editable/?utm_source=chatgpt.com

² ⁴ ³⁶ ⁴⁰ Tiptap Docs

<https://tiptap.dev/docs>

³ Editing engine | CKEditor 5 Framework Documentation

<https://ckeditor.com/docs/ckeditor5/latest/framework/architecture/editing-engine.html>

5 8 13 35 **Extensions | Tiptap Editor Docs**

<https://tiptap.dev/docs/editor/extensions/overview>

6 27 28 **Track changes overview | CKEditor 5 Documentation**

<https://ckeditor.com/docs/ckeditor5/latest/features/collaboration/track-changes/track-changes.html>

7 **Toolbar Pattern | APG | WAI**

https://www.w3.org/WAI/ARIA/apg/patterns/toolbar/?utm_source=chatgpt.com

9 21 25 **ProseMirror Guide**

<https://prosemirror.net/docs/guide/>

10 **Schema | CKEditor 5 Framework Documentation**

<https://ckeditor.com/docs/ckeditor5/latest/framework/deep-dive/schema.html>

11 38 **Clipboard API - Web APIs | MDN**

https://developer.mozilla.org/en-US/docs/Web/API/Clipboard_API

12 **Format Painter | TinyMCE Documentation**

<https://www.tiny.cloud/docs/tinymce/5/enterprise-formatpainter/>

14 **Intl.Segmenter - JavaScript - MDN Web Docs - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Intl/Segmenter?utm_source=chatgpt.com

15 **ProseMirror Guide**

https://prosemirror.net/docs/guide/?utm_source=chatgpt.com

16 29 39 **Mammoth .docx to HTML converter**

https://github.com/mwilliamson/mammoth.js/?utm_source=chatgpt.com

17 **Using Web Workers - Web APIs - MDN Web Docs**

https://developer.mozilla.org/en-US/docs/Web/API/Web_Workers_API/Using_web_workers?utm_source=chatgpt.com

18 34 **Pages | Tiptap Pages**

<https://tiptap.dev/docs/pages/getting-started/overview>

19 **hyphens CSS property - MDN Web Docs - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/hyphens?utm_source=chatgpt.com

20 **Pointer Events**

https://w3c.github.io/pointerevents/?utm_source=chatgpt.com

22 **Introduction | Lexical**

<https://lexical.dev/docs/intro>

23 **Normalizing**

https://docs.slatejs.org/concepts/11-normalizing?utm_source=chatgpt.com

24 **useSyncExternalStore**

https://react.dev/reference/react/useSyncExternalStore?utm_source=chatgpt.com

26 **Yjs Docs: Introduction**

https://docs.yjs.dev/?utm_source=chatgpt.com

30 **PagedJS**

https://github.com/pagedjs/pagedjs/?utm_source=chatgpt.com

31 **Cross Site Scripting Prevention Cheat Sheet**

https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html?utm_source=chatgpt.com

32 **File Upload - OWASP Cheat Sheet Series**

https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

33 **Secure contexts - MDN Web Docs - Mozilla**

https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Secure_Contexts?utm_source=chatgpt.com

37 **Editor State | Lexical**

<https://lexical.dev/docs/concepts/editor-state>